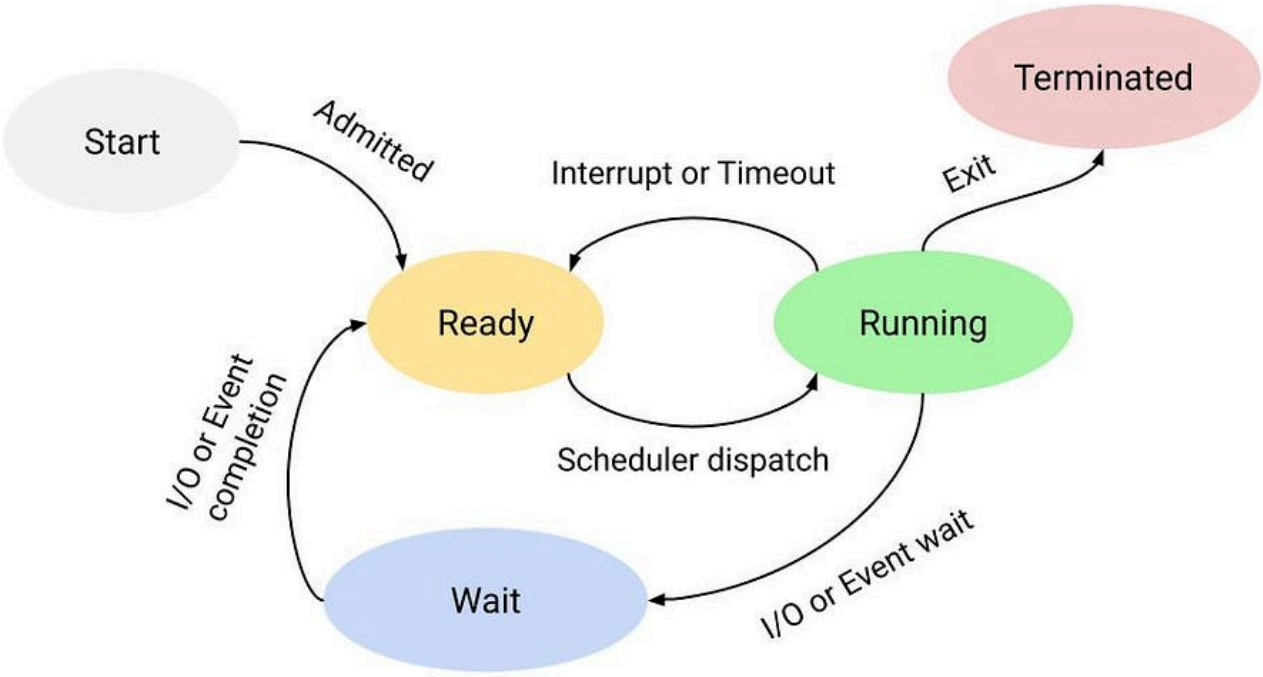
Lesson 5

What is scheduling?

- CPU scheduling is a process used by the operating system to decide which task or process gets to use the CPU at a particular time.
- CPU scheduling mainly considers the ready, running and wait stages of the process management cycle.
- The main function of CPU scheduling is to ensure that whenever the CPU remains idle, the OS has at least selected one of the processes available in the ready-to-use line.



Single processor vs Multi-Processor Needs / Advantages of CPU scheduling

- Improve CPU utilization
- Increase system performance
- Reduce waiting time
- Execute processes efficiently

Importance of the waiting stage

- Maximum CPU utilization
- Time management

CPU, I/O burst cycle

CPU Burst

- **At its core, a CPU burst is a period during which a process or program demands and actively utilizes the Central Processing Unit (CPU) for computation**. It's when the CPU executes a sequence of instructions for a specific task. These bursts are not malicious, to the contrary, they are essential for processing tasks and computations in virtually every application, from running operating systems to handling complex software.
- **CPU bursts have two typical characteristics – duration and variability.**
- CPU bursts can vary significantly in duration. Some may be short and intensive, while others may be long and less resource-demanding. Understanding the duration of CPU bursts helps in resource allocation and task scheduling.
- The variability of CPU bursts is a critical factor in system performance. Burst patterns can be irregular, leading to unpredictable system behaviour. Managing this variability is essential for maintaining system stability.
- **Long CPU bursts are mostly common in CPU-intensive computing scenarios.** Some applications that showcase big CPU bursts are video

rendering and transcoding, scientific simulations and modelling, compiling large software projects or cryptographic operations.

- **Identifying the length and variability of CPU bursts is an important step when doing performance optimization in systems.** To effectively manage these, tools and metrics are essential. Profiling tools like that can help monitor CPU usage (for example, perf program on Linux). Key metrics include CPU utilization, execution time, and context switches. Analyzing these metrics provides insights into how efficiently CPU resources are utilized.

I/O Burst

- In contrast to CPU bursts, I/O bursts involve Input/Output operations. During an I/O burst, **a process or program waits for data to be read from or written to external storage devices, such as disks or network resources.** I/O bursts are common in scenarios where data needs to be fetched from or delivered to external sources.
- **I/O bursts also have two typical characteristics, waiting time and I/O operation type:**
  - I/O bursts are characterized by the time a process waits for data to be retrieved or stored. This waiting time can vary widely, depending on the speed of the storage medium and the complexity of the I/O operation. For example, it's much faster to bring data from a cache memory than to go all the way to a hard drive.
  - I/O bursts can involve various types of operations, including disk I/O (reading/writing files), network I/O (sending/receiving data over a network), and more. Each type has its unique characteristics and challenges.
- **As we'd expect, there exist workloads that are more I/O-intensive and, therefore, require more I/O bursts.** Those may include things like database operations, where data is retrieved or stored on disk, web servers handling numerous client requests or video streaming, where data is constantly read from storage (or network).

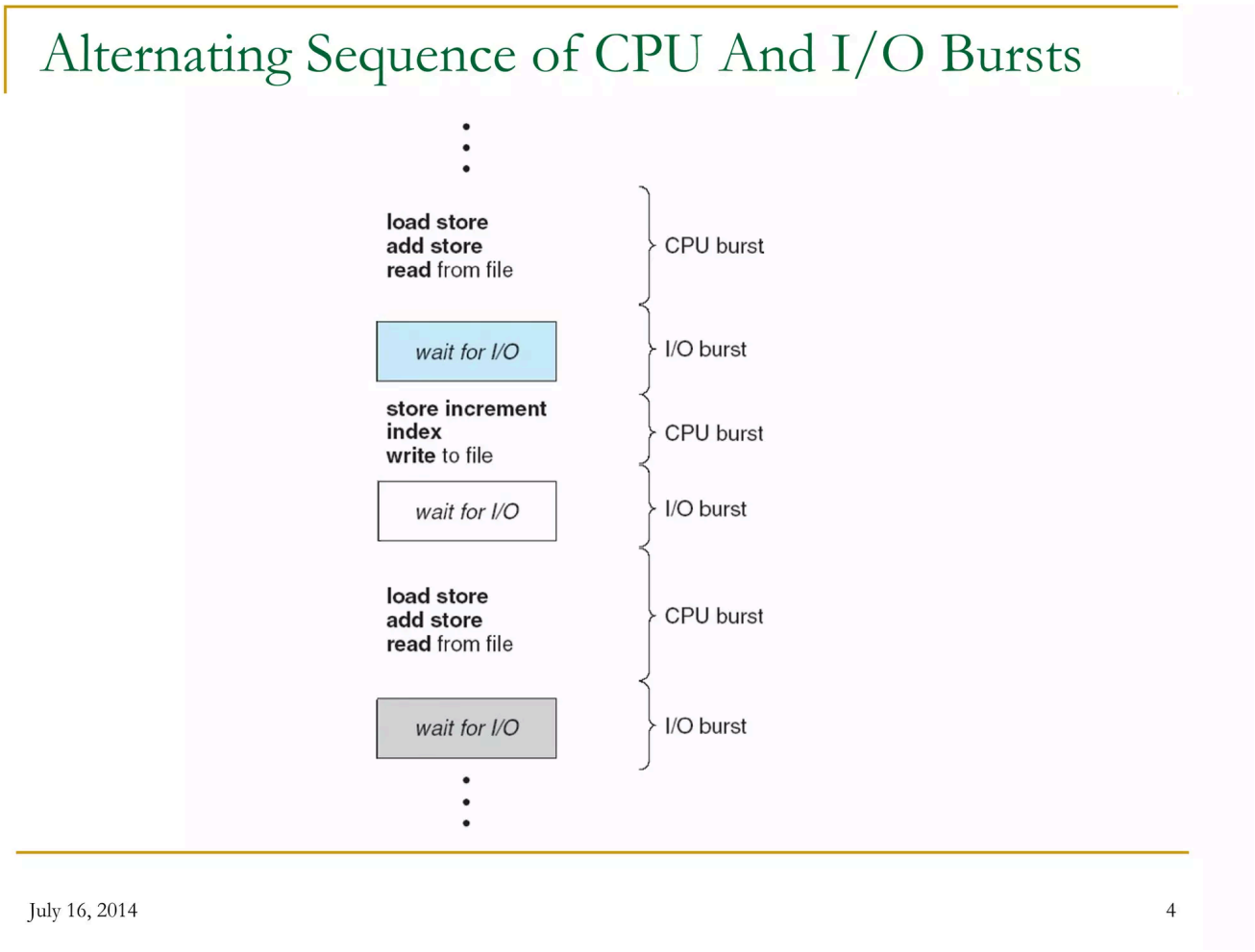
- **To optimize I/O performance, it's crucial to measure and analyze I/O bursts.** Tools like `iostat` and `sar` on Linux provide insights into disk I/O. Metrics include disk throughput, I/O queue length, and response times. By monitoring these metrics, system administrators can identify and address performance bottlenecks.

Relationship Between CPU and I/O Bursts

- **In many real-world scenarios, CPU and I/O bursts are interconnected.** For example, a web server may experience CPU bursts while processing requests and I/O bursts when reading or writing data to storage. Balancing the utilization of CPU and I/O resources is essential for optimizing system performance.
- **Efficiently managing both CPU and I/O-intensive workloads involves strategic resource allocation.** Techniques like task scheduling and parallel processing can help balance these workloads. For example, multi-threading allows CPU-bound tasks to continue execution while waiting for I/O operations to complete.
- Real-world scenarios where this relationship plays a critical role in the performance of systems include the following:
  - Database management – database systems often face a delicate balance between CPU-intensive query processing and I/O-bound data retrieval. Optimizations, such as indexing and caching, are used to minimize the impact of bursts on database performance
  - Web server performance – web servers encounter bursts of incoming requests that require CPU processing. Additionally, serving static assets like images or videos involves I/O bursts. Efficient request handling and caching strategies are crucial in maintaining responsive web services

Example of CPU-I/O Burst Cycle:

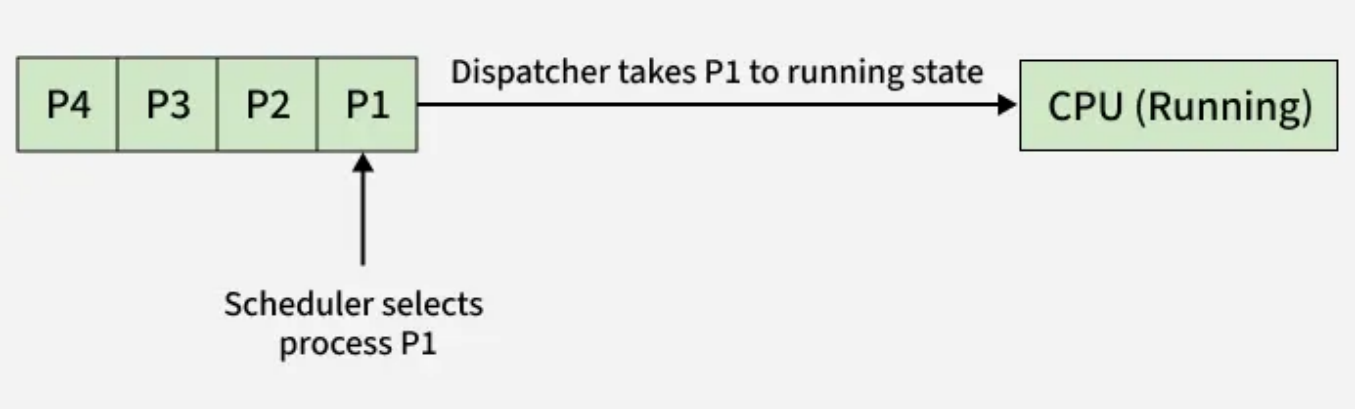
Time	Process State	Activity
0-3	CPU Burst	Executing calculations
3-7	I/O Burst	Waiting for disk read
7-12	CPU Burst	Processing data
12-16	I/O Burst	Writing output to file



Dispatcher

Once the Short Term Scheduler selects the next process to execute, the Dispatcher takes over. The Dispatcher is a small, specialized program that gives control of the CPU to the process chosen by the short-term scheduler. It performs the low-level work needed to actually start executing the selected

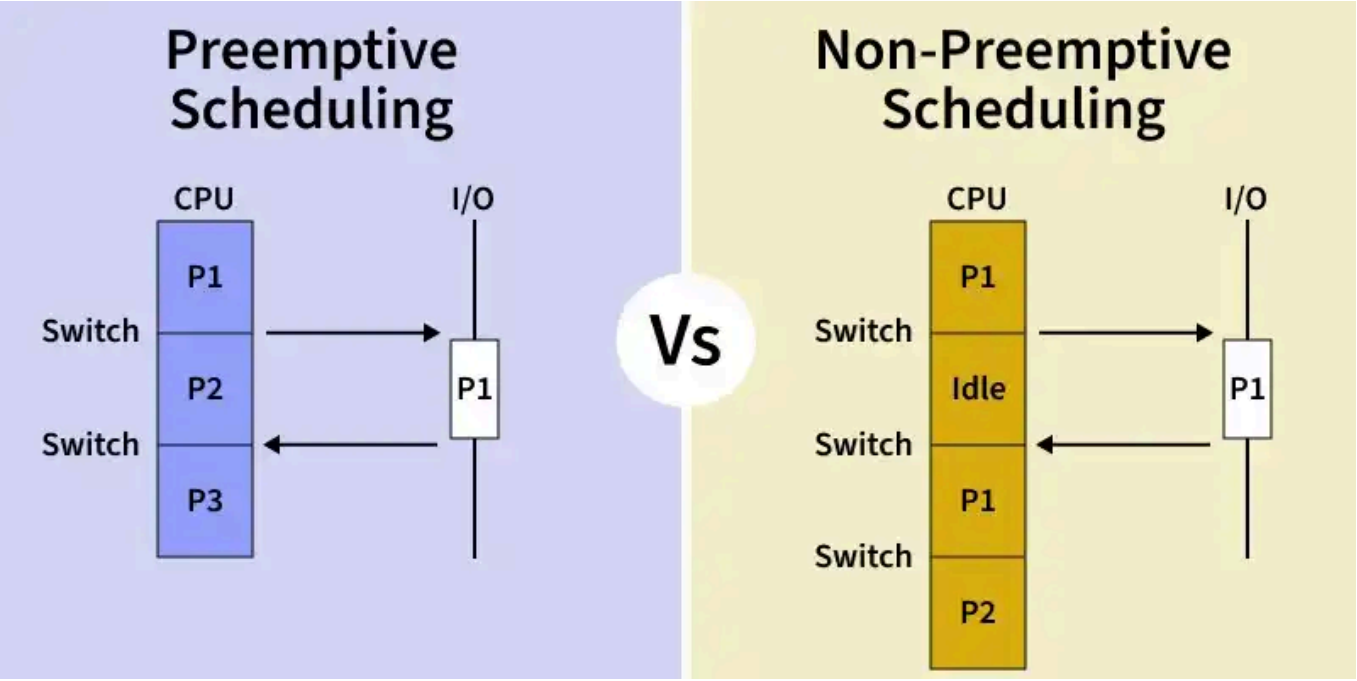
process..



Difference Between Scheduler and Dispatcher

Scheduler	Dispatcher
Decides which process should be executed next.	Transfers control of CPU to the process selected by the scheduler.
To select the process and determine execution order.	To start the execution of the selected process.
Long-term, Medium-term and Short-term.	No types; it's a single module.
Works independently.	Dependent on the scheduler's decision.
Uses algorithms like FCFS, SJF, RR, Priority, etc.	No specific algorithm used.
Negligible and occurs less frequently.	Time taken is known as Dispatch Latency.
Process selection and queue management.	Context switching, mode change and process start.
Works with the ready queue and dispatcher.	Works with CPU and the selected process.
Takes longer than the dispatcher.	Executes in a very short time.

Preemptive vs Non-preemptive scheduling



Preemptive Scheduling

- In preemptive scheduling, the **operating system can interrupt a running process to allocate the CPU to another process usually due** to priority rules or time-sharing policies. A process may be moved from Running → Ready state before it finishes.
- **Examples:**

  - Round Robin
  - Shortest Remaining Time First (SRTF)
  - Priority Scheduling (preemptive version)

Advantages

- Prevents one process from monopolizing CPU
- Better average response time in multi-user systems
- Widely used in modern OS (Windows, Linux, macOS)

Disadvantages

- More complex to implement
- Higher overhead from context switching

- Can cause starvation of low-priority processes
- Risk of concurrency issues if preempted during shared resource access

Non-Preemptive Scheduling

- In non-preemptive scheduling, **once a process starts using the CPU, it runs until it finishes or moves to a waiting state.** The OS cannot forcibly take away the CPU.
  Examples:

> - First Come First Serve (FCFS)
> - Shortest Job First (SJF)
> - Priority Scheduling (non-preemptive version)

Advantages

- It is easy to implement in an operating system. It was used in Windows 3.11 and early macOS.
- It has a minimal scheduling burden.
- Less computational resources are used.

Disadvantages

- It is open to denial of service attack. A malicious process can take CPU forever.
- Since we cannot implement round robin, the average response time becomes less.

Preemptive Scheduling	Non-Preemptive Scheduling
In this resources(CPU Cycle) are allocated to a process for a limited time.	Once resources(CPU Cycle) are allocated to a process, the process holds it till it completes its burst time or switches to waiting state
Process can be interrupted in between.	Process can not be interrupted until it terminates itself or its time is up
If a process having high priority frequently arrives in the ready queue, a	If a process with a long burst time is running CPU, then later coming process with less CPU burst

Preemptive Scheduling	Non-Preemptive Scheduling
low priority process may starve	time may starve
It has overheads of scheduling the processes	It does not have overheads
Average process response time is less	Average process response time is high
Decisions are made by the scheduler and are based on priority and time slice allocation	Decisions are made by the process itself and the OS just follows the process's instructions
More as a process might be preempted when it was accessing a shared resource.	Less as a process is never preempted.
Examples of preemptive scheduling are Round Robin and Shortest Remaining Time First	Examples of non-preemptive scheduling are First Come First Serve and Shortest Job First

- CPU scheduling happens at;
  1. Running to waiting
  2. Running to ready
  3. Waiting to ready
  4. Terminate
- 1 and 4 are non-preemptive
- 2 and 3 are preemptive
- **Preemptive Scheduling:** Preemptive scheduling is used when a process switches from running state to ready state or from the waiting state to the ready state.
- **Non-Preemptive Scheduling:** Non-Preemptive scheduling is used when a process terminates , or when a process switches from running state to waiting state.

Scheduling criteria

- **CPU Utilization:** The main purpose of any CPU algorithm is to keep the CPU as busy as possible. Theoretically, CPU usage can range from 0 to 100 but in a real-time system, it varies from 40 to 90 percent depending on the system load.

- **Throughput:** The average CPU performance is the number of processes performed and completed during each unit. This is called throughput. The output may vary depending on the length or duration of the processes.
- **Turn Round Time:** For a particular process, the important conditions are how long it takes to perform that process. The time elapsed from the time of process delivery to the time of completion is known as the conversion time. Conversion time is the amount of time spent waiting for memory access, waiting in line, using CPU and waiting for I/O.
- **Waiting Time:** The Scheduling algorithm does not affect the time required to complete the process once it has started performing. It only affects the waiting time of the process i.e. the time spent in the waiting process in the ready queue.
- **Response Time:** In a collaborative system, turn around time is not the best option. The process may produce something early and continue to computing the new results while the previous results are released to the user. Therefore another method is the time taken in the submission of the application process until the first response is issued. This measure is called response time.